



GPU Teaching Kit

Accelerated Computing



Lecture 2.5 – Nsight Compute and Nsight Systems

Introduction to the CUDA Toolkit

Objective

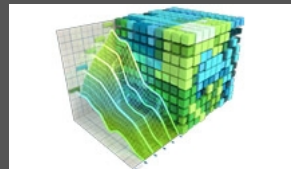
- To become familiar with Nsight Systems and Compute

Profiling Tools

nvprof

```
==20050== Profiling result:
Time(s)      Time      Calls      Avg      Min      Max      Name
49.88s  866.69ms  504758  1.7170us  1.5040us  2.0160us  void th
25.33s  440.60ms  252602  1.7410us  1.5360us  2.3680us  void th
t, thrust::detail::device_generate_function<thrust::detail::fill_fun
17.07s  206.69ms   200      1.403ms  1.000ms  1.723ms  kerComp
2.98s   51.81ms   200      259.69us  246.97us  264.83us  kerPoke
1.16s   20.17ms   501      40.265us  928ns  17.677ms  [CUDA p
0.93s   10.15ms   200      86.991us  71.840us  90.751us  kerGall
0.73s   12.53ms   400      31.589us  14.720us  59.432us  [CUDA w
0.69s   12.07ms   200      60.370us  59.680us  62.304us  kerRenaj
0.63s   10.99ms   200      54.963us  52.000us  58.200us  kerPoke
0.32s   5.552ms    200      27.761us  22.559us  33.152us  [CUDA w
0.12s   2.134ms     1      2.1342ms  2.1342ms  2.1342ms  void th
```

NVVP

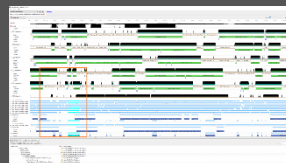


Phasing out

**Nsight
Compute**



**Nsight
Systems**



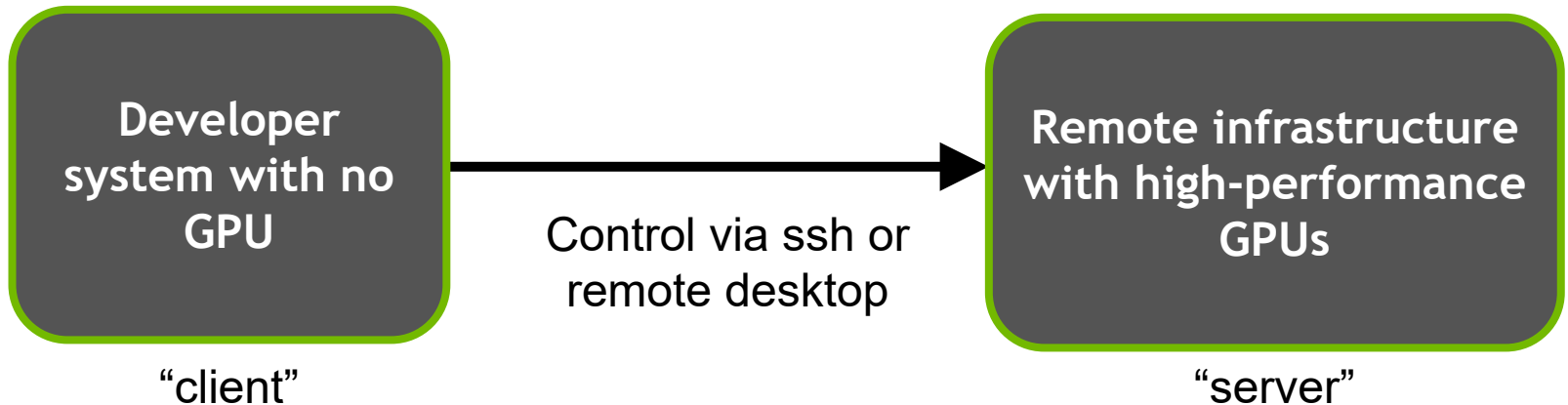
Current

See lecture 2-4 for an overview of all tools

Nsight Compute & Nsight Systems

- Command Line and Interactive Profilers
- Bundled with CUDA Toolkit
- Newer standalone downloads available on NVIDIA website
- Nsight Systems
 - “Feeds and speeds:” getting data into the GPU and profiling GPU utilization
- Nsight Compute
 - Kernel-level profiling

A Common GPU Development Model



Two-Phase Profiling

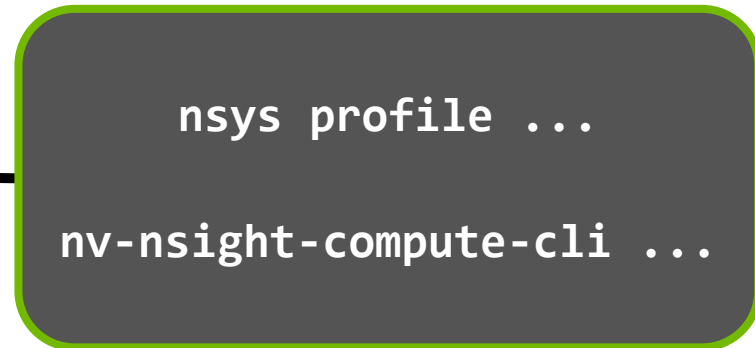
- (interactive profiling also supported)

“client”



Analyze profiling data

“server”



Record profiling data

scp or shared file
system



Before Profiling

- Options to improve your profiling experience
 - Host code annotations with Nvidia Tools Extension Library
 - Correctness & cuda-memcheck
 - Compilation flags
 - Ensure Nsight system environment is correct

Before Profiling: Host Code Annotations

`#include <nvToolsExt.h>` and link with `-lnvToolsExt`

Will show up as a named span in the Nsight System GUI

Useful for marking parts of the code for later reference.

```
nvtxRangePush("sleeping");  
sleep(100);  
nvtxRangePop();
```


Before Profiling: cuda-memcheck

- Certain kinds of errors cause CUDA programs to complete, but crash under profiling
- Check your program with cuda-memcheck if code behaves incorrectly under profiling

```
cuda-memcheck ./my-program ...
```

Before Profiling: Compilation Flags

- Compile device code with optimizations
 - Optimizations dramatically change performance
 - Remove “-G” device-debug flag from nvcc
- Compile device code with line information
 - Minimal information included in binary to map PTX/SASS to source code
 - Add “-lineinfo” flag to nvcc

`nvcc -G main.cu` → `nvcc -lineinfo main.cu`

Before Profiling: Nsight Systems Configuration

Nsight System uses various system hooks to accomplish profiling.

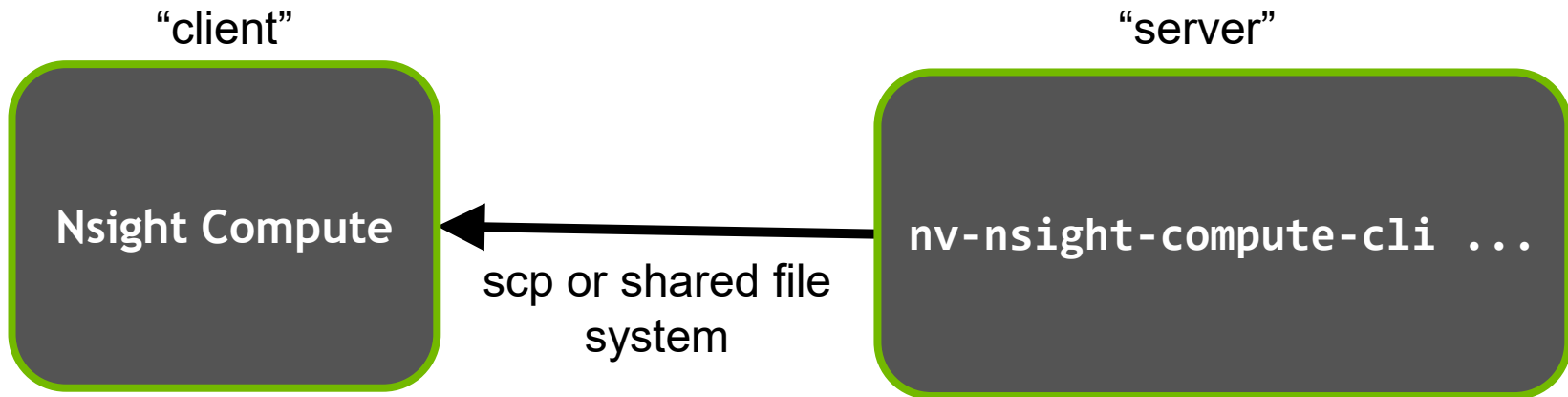
Some errors would reduce the amount or accuracy of gathered info, some will make system profiling impossible. An example of a GOOD output: (check with `nsys status -e`)

```
$ nsys status -e
>> Sampling Environment Check
>> Linux Kernel Paranoid Level = 2: OK
>> Linux Distribution = Ubuntu
>> Linux Kernel Version = 4.16.15-41615: OK
>> Linux perf_event_open syscall available: OK
>> Sampling trigger event available: OK
>> Intel(c) Last Branch Record support: Available
>> Sampling Environment: OK
```

Consult the Nsight Systems documentation or your system administrator to correct any issues.

Nsight Compute

- Record and analyze detailed kernel performance metrics
- Two interfaces:
 - GUI (nv-nsight-cu)
 - CLI (nv-nsight-cu-cli)
- Directly consuming 1000 metrics is challenging, we use the GUI to help
- Use a two-part record-then-analyze process



Performance Counters

- Device has many performance counters to record detailed information
 - Made available as “metrics”.
- Nsight Compute helps you interpret these

```
$ nv-nsight-cu-cli --devices 0 --query-metrics
```

```
lts__t_sectors_srcunit_l1_op_atom_dot_cas  
l1tex__data_pipe_lsu_wavefronts_mem_shared_cmd_write  
lts__t_sectors_srcunit_l1_aperture_sysmem_op_read  
lts__t_requests_op_red_lookup_hit  
lts__t_sectors_equiv_l1tagmiss_pipe_tex_mem_texture_op_ld  
l1tex__t_bytes_pipe_tex_lookup_miss  
l1tex__texin_requests_mem_texture  
l1tex__t_bytes_pipe_lsu_mem_local_op_ld_lookup_miss  
l1tex__t_bytes_pipe_tex_mem_surface_op_red_lookup_miss  
...
```

Record Kernel Traces

- Recording may be for the whole execution, or usually restricted to a single launch of a kernel

```
$ nv-nsight-cu-cli      \ <nsight compute cli>
  --kernel-id ::mygemm:6 \ <6th launch of “mygemm” kernel>
  --section “.”         \ <output file name>
  -o output_file        \ <executable to run>
  executable
```

Open Nsight Compute

- We will import the recorded file

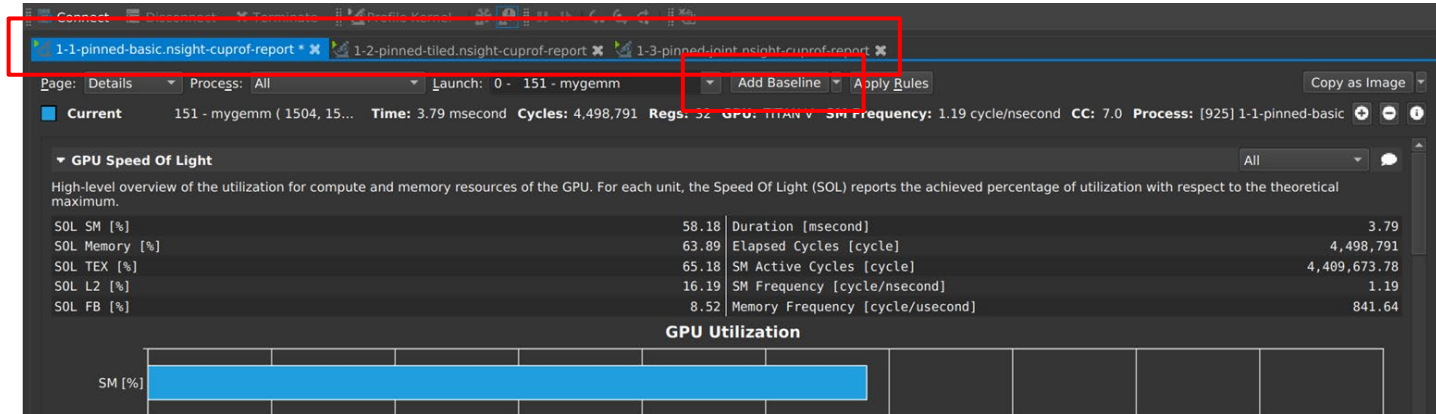
File > Open File ... > output_file.nsisight-cuprof-report

- We can open multiple files in multiple tabs, if desired

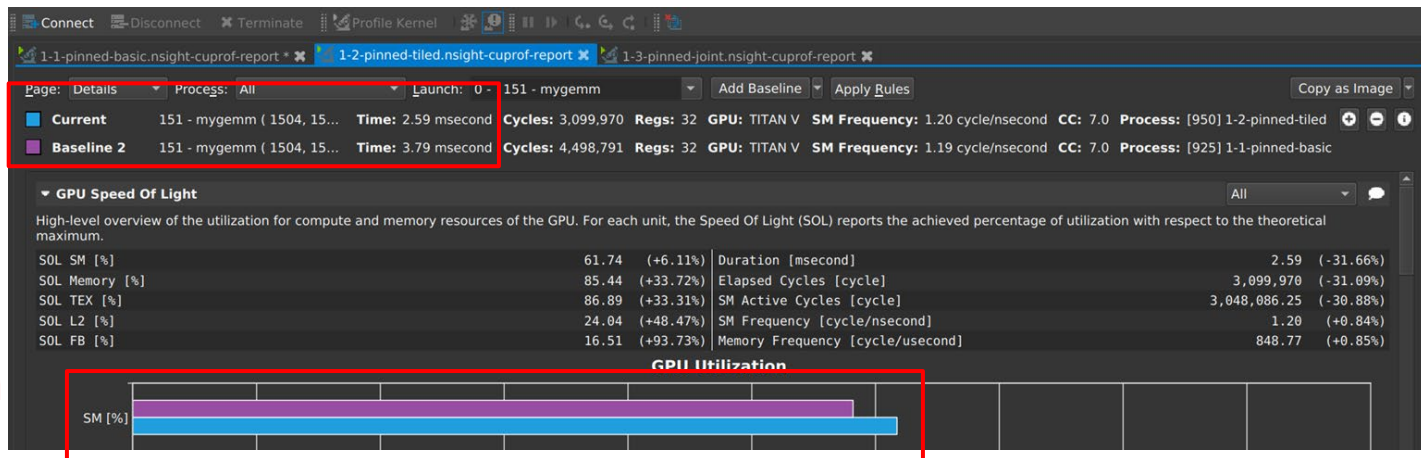
First Look

- Can compare multiple open codes with baseline button
 - For comparing effect of optimizations

Tabs and
baseline
button

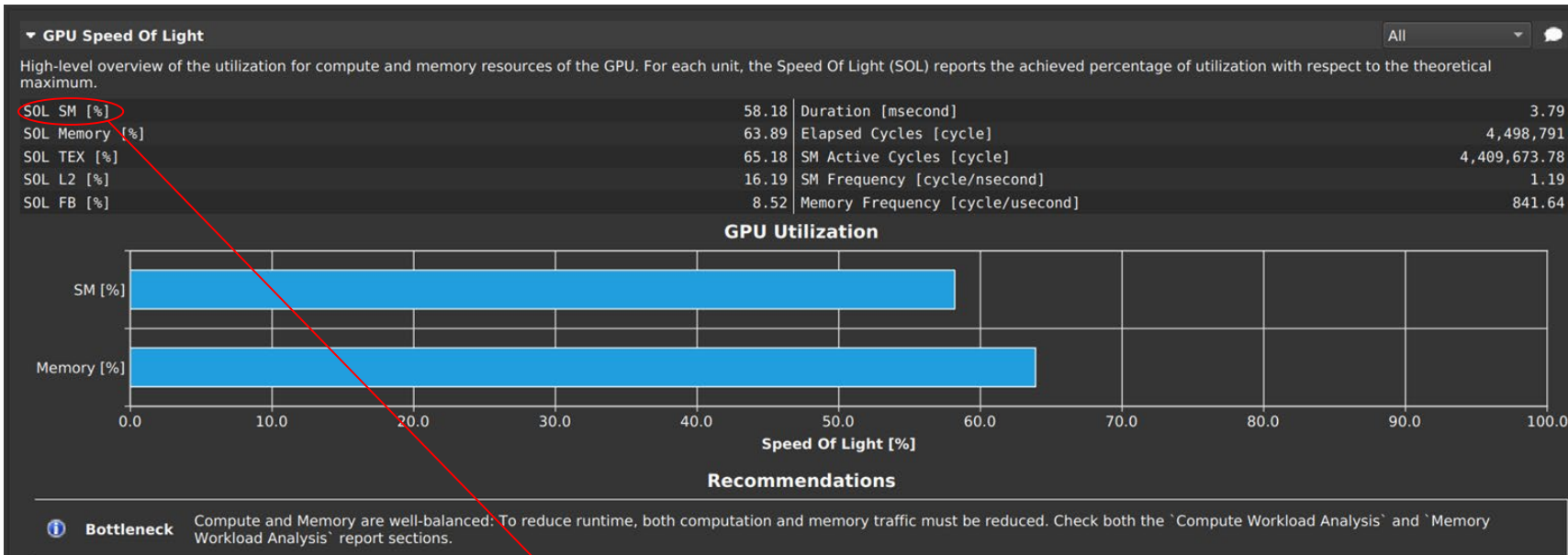


Next tab
now has
comparison



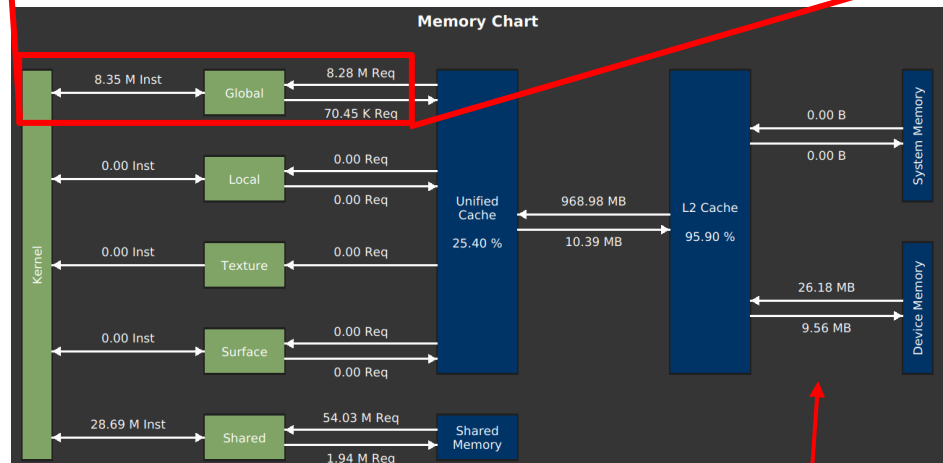
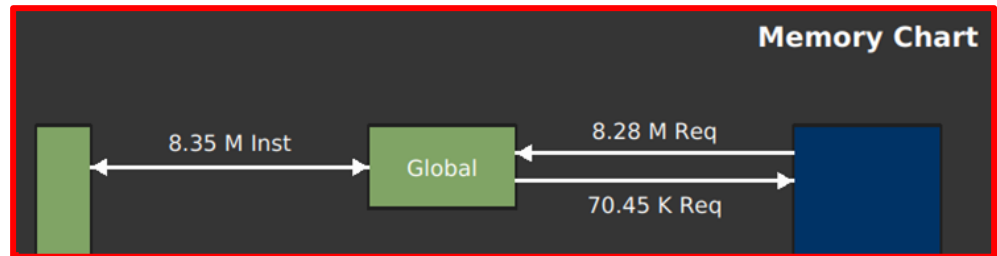
GPU Speed of Light

- Utilization compared to theoretical maximums



Tip: mouse over each to see the associated metric

Workload Memory Analysis Chart



Executed
instructions that
reference a memory
space →

Requests to
the memory

Amount of
data moving

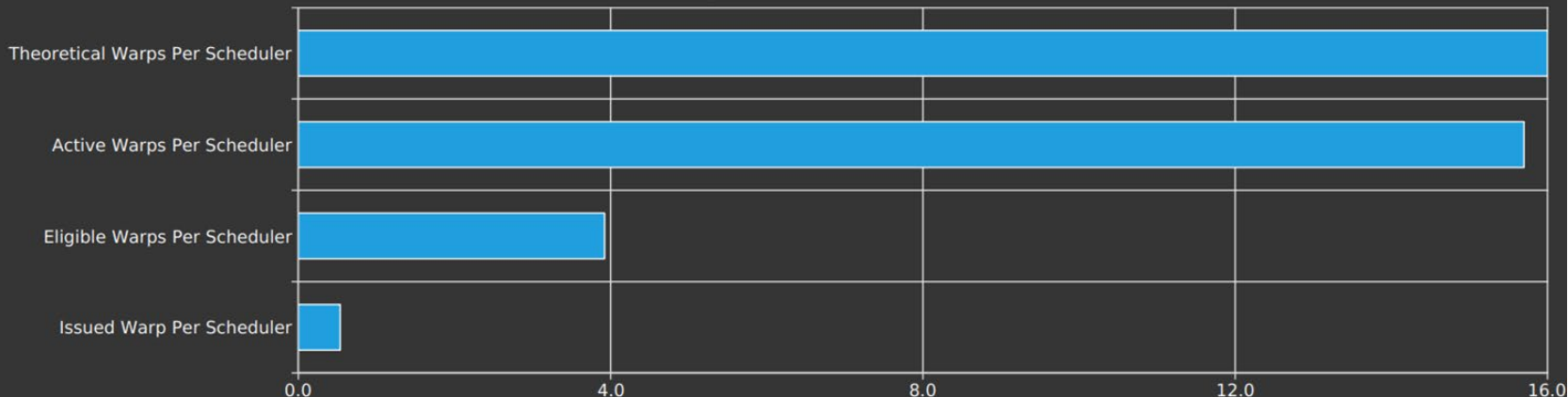
Scheduler Statistics

- Activity of the scheduler issuing instructions.
 - Maximum possible warps per scheduler
 - Warps that have not exited a kernel
 - Warps that have execution dependencies satisfied
 - Warps actually issued

Summary of the activity of the schedulers issuing instructions. Each scheduler maintains a pool of warps that it can issue instructions for. The upper bound of warps in the pool (Theoretical Warps) is limited by the launch configuration. On every cycle each scheduler checks the state of the allocated warps in the pool (Active Warps). Active warps that are not stalled (Eligible Warps) are ready to issue their next instruction. From the set of eligible warps the scheduler selects a single warp from which to issue one or more instructions (Issued Warp). On cycles with no eligible warps, the issue slot is skipped and no instruction is issued. Having many skipped issue slots indicates poor latency hiding.

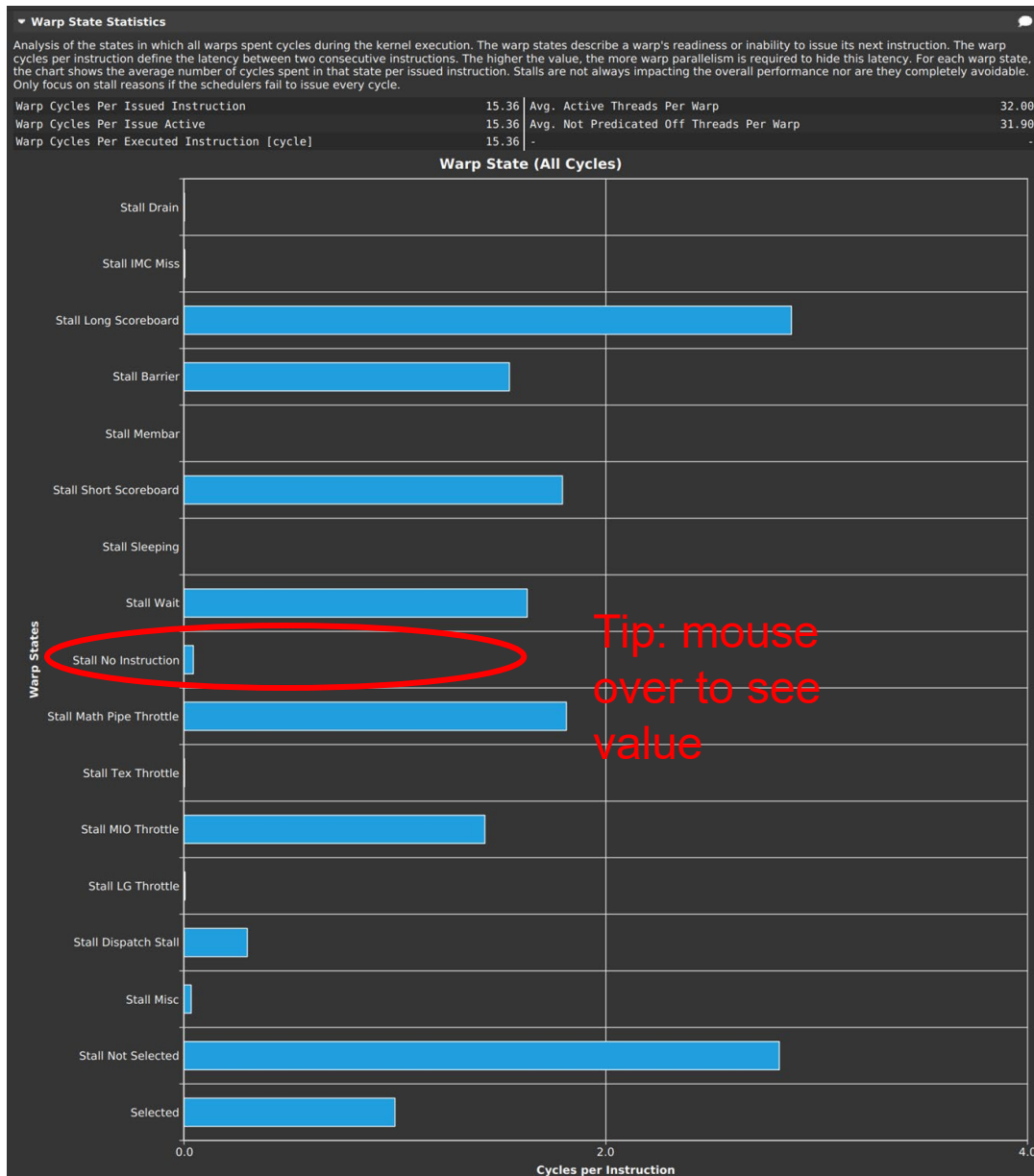
Active Warps Per Scheduler [warp]	15.70	Instructions Per Active Issue Slot [inst/cycle]	1
Eligible Warps Per Scheduler [warp]	3.92	No Eligible [%]	46.25
Issued Warp Per Scheduler	0.54	One or More Eligible [%]	53.75

Warps Per Scheduler



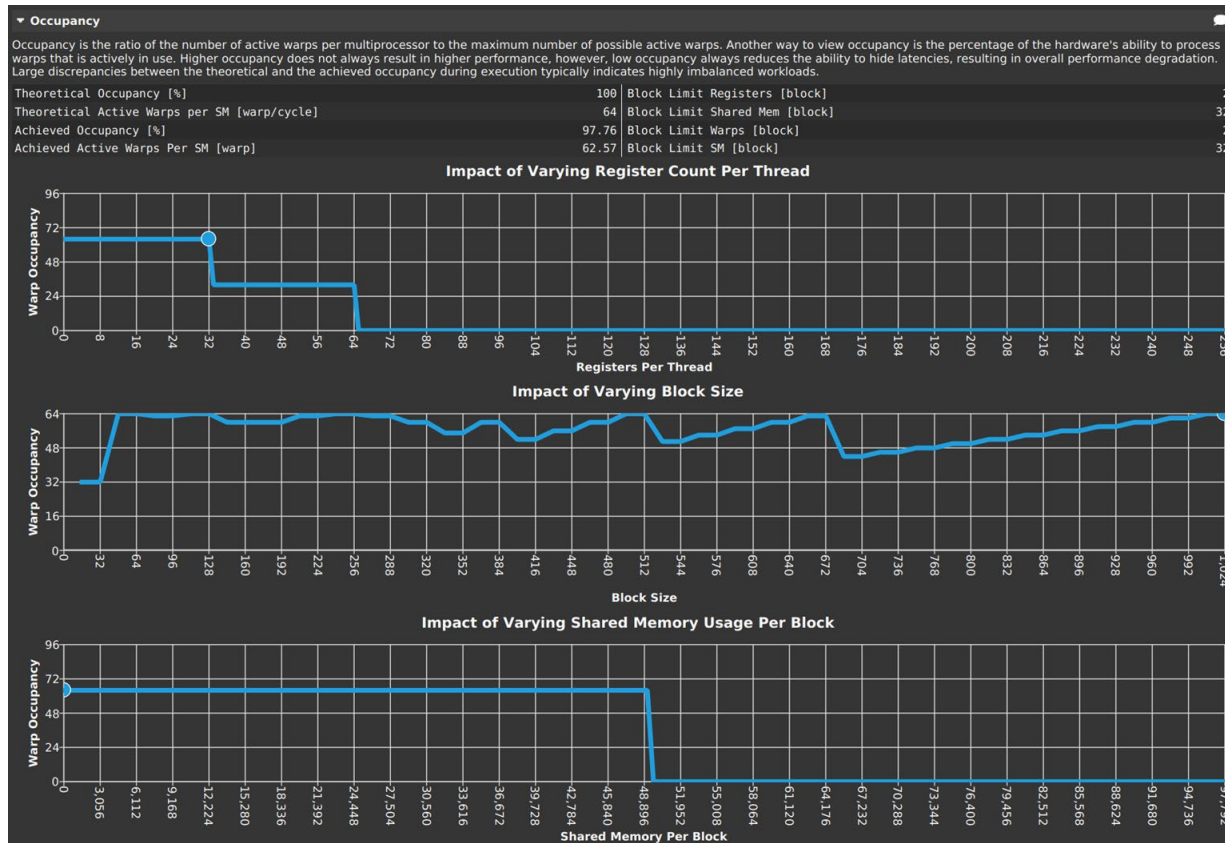
Warp State Statistics

- How much time warps spend in each state



Occupancy

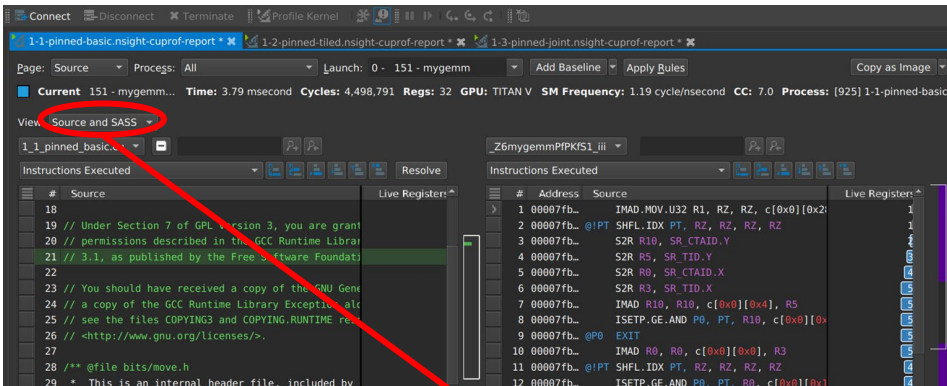
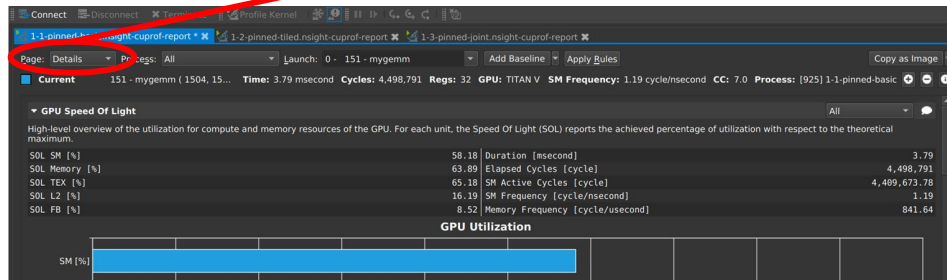
- How many warps are active compared to the maximum possible
 - Achieved: true number of active warps as average
 - Charts show how kernel resources and grid dimension affect occupancy



Instruction Hotspots

- Show metrics by source line, PTX instruction, or SASS instruction

Switch page to “Source”



“Source and PTX” (usually) or “Source and SASS”

Instruction Hotspots

- Source file may not load if profiling was recorded on another system

The screenshot displays the NVIDIA NSIGHT Systems interface. At the top, there are tabs for different reports: '1-1-pinned-basic.nsigth-cuprof-report', '1-2-pinned-tiled.nsigth-cuprof-report', and '1-3-pinned-joint.nsigth-cuprof-report'. Below the tabs, a status bar shows 'Current 151 - mygemm...', 'Time: 3.79 msecond', 'Cycles: 4,498,791', 'Regs: 32', 'GPU: TITAN V', 'SM Frequency: 1.19 cycle/nsecond', 'CC: 7.0', and 'Process: [925] 1-1-pinned-basic'. The 'View' dropdown is set to 'Source and SASS'. The left panel shows the source code for '1_1_pinned_basic.cu' with a 'Resolve' button circled in red. The right panel shows the assembly instructions for '_Z6mygemmPfPKfS1_iii'.

#	Source	Live Register
18		
19	// Under Section 7 of GPL version 3, you are granted	
20	// permissions described in the GCC Runtime Library	
21	// 3.1, as published by the Free Software Foundation	
22		
23	// You should have received a copy of the GNU General	
24	// a copy of the GCC Runtime Library Exception along	
25	// see the files COPYING3 and COPYING.RUNTIME respectively	
26	// <http://www.gnu.org/licenses/>.	
27		
28	/** @file bits/move.h	
29	* This is an internal header file, included by	

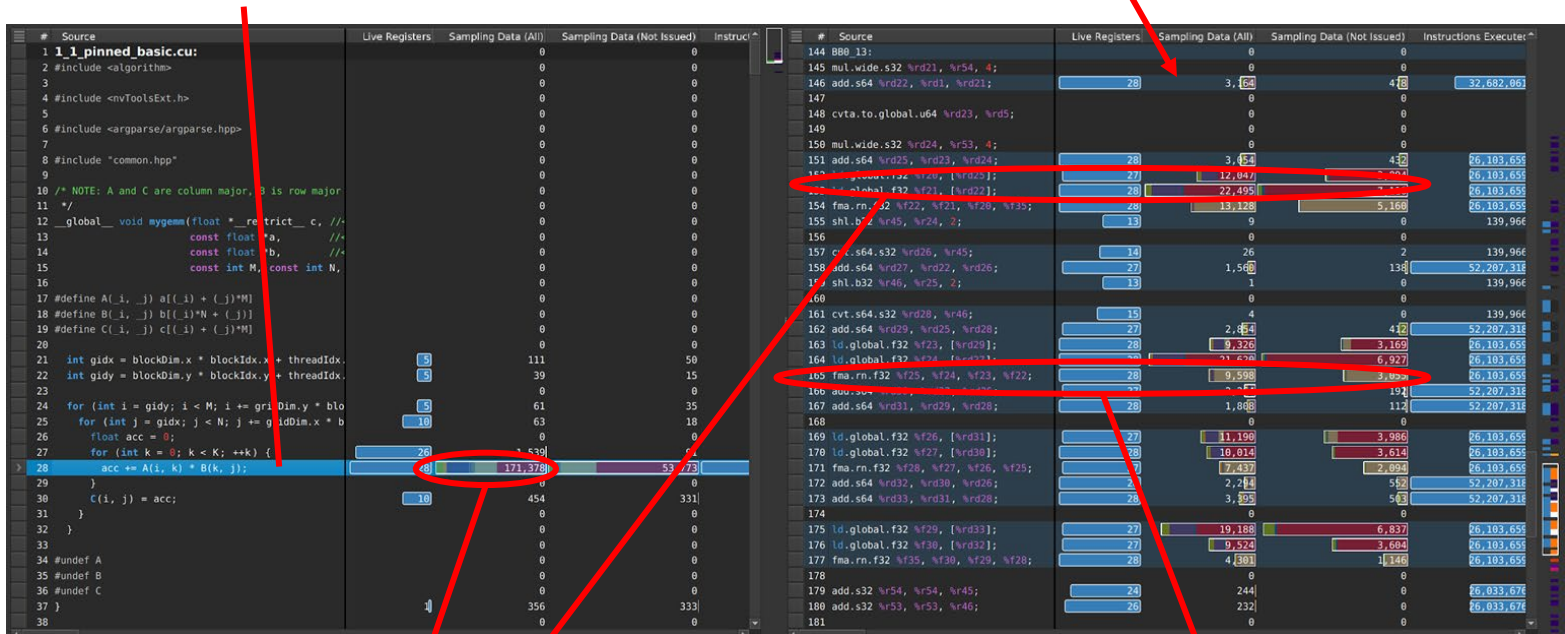
#	Address	Source	Live Register
1	00007fb...	IMAD.MOV.U32 R1, RZ, RZ, c[0x0][0x2]	1
2	00007fb...	@!PT SHFL.IDX PT, RZ, RZ, RZ, RZ	1
3	00007fb...	S2R R10, SR_CTAID.Y	1
4	00007fb...	S2R R5, SR_TID.Y	1
5	00007fb...	S2R R0, SR_CTAID.X	4
6	00007fb...	S2R R3, SR_TID.X	5
7	00007fb...	IMAD R10, R10, c[0x0][0x4], R5	5
8	00007fb...	ISETP.GE.AND P0, PT, R10, c[0x0][0x...	5
9	00007fb...	@P0 EXIT	5
10	00007fb...	IMAD R0, R0, c[0x0][0x0], R3	5
11	00007fb...	@!PT SHFL.IDX PT, RZ, RZ, RZ, RZ	4
12	00007fb...	ISETP.GE.AND P0, PT, R0, c[0x0][0x...	4

Click “resolve” and find your local copy of the code that was compiled or run remotely.

Instruction Hotspots

Click a line to highlight lines
from other side...

...corresponding PTX/SASS
lines over here.

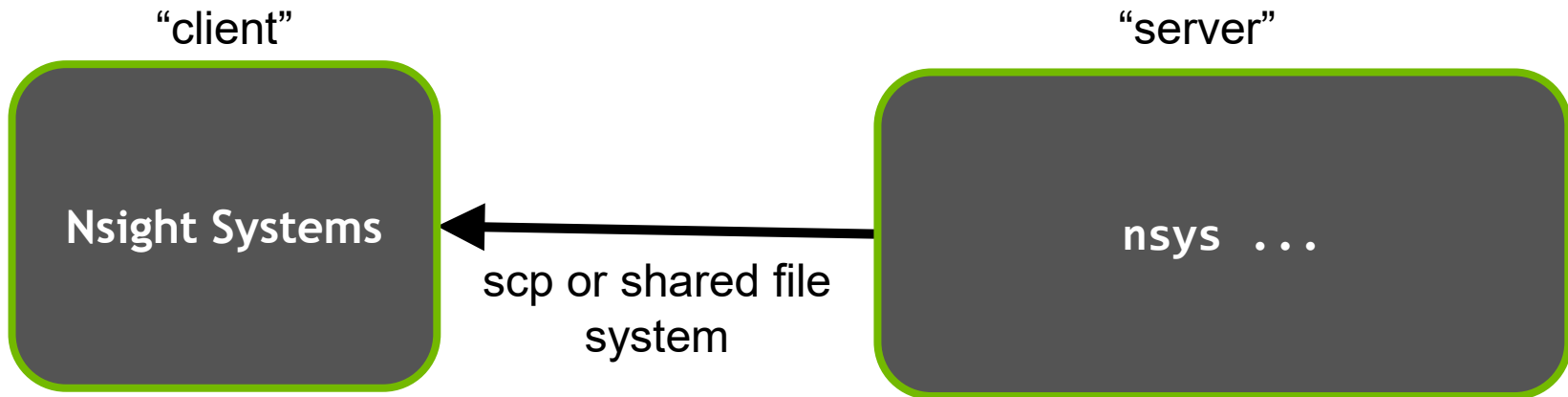


Program counter spends most of its
time on instructions from this line.
Mouse over for breakdown.

Sometimes, stalls can show
in a following instruction that
depends on a previous one

Nsight Systems

- Record and analyze system utilization information
- Two interfaces:
 - GUI (nsight-sys)
 - CLI (nsys)
- Use a two-part record-then-analyze process



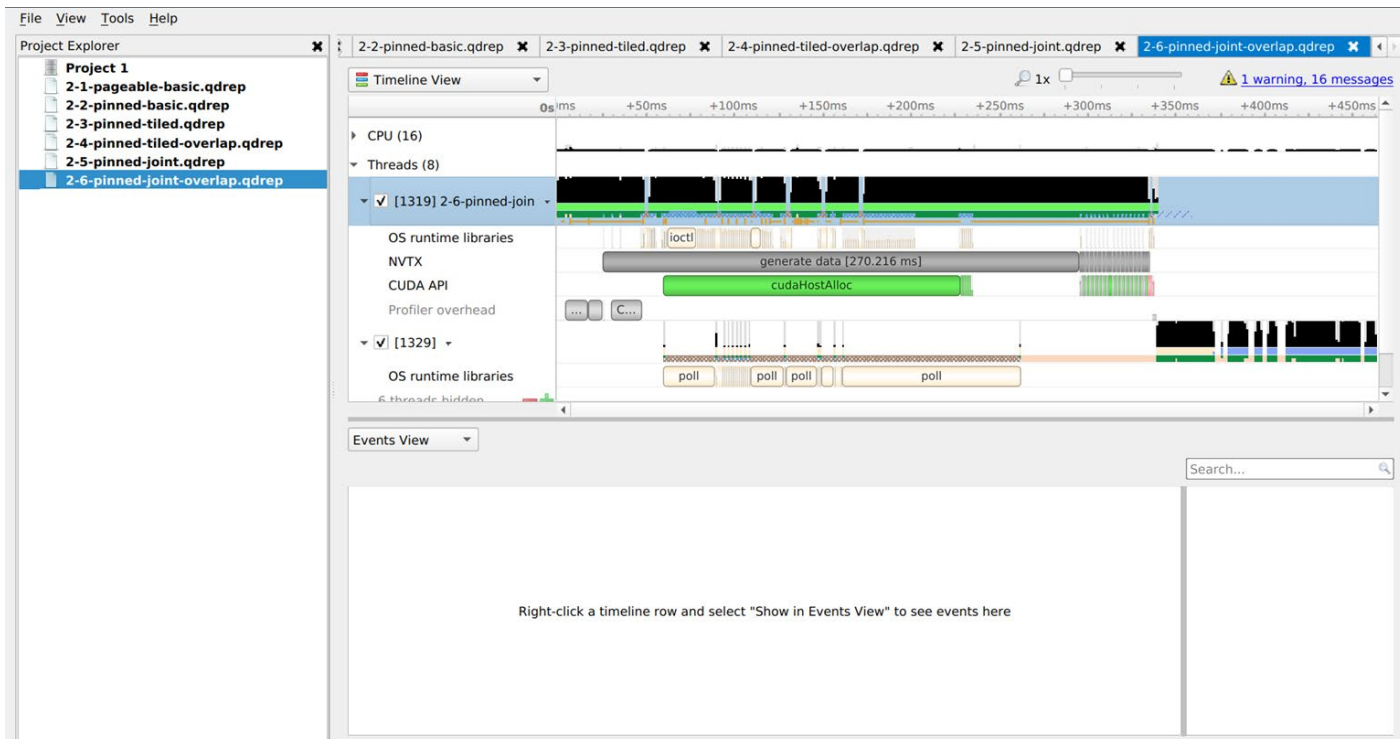
Record Execution Traces

- Record system information for the entire execution

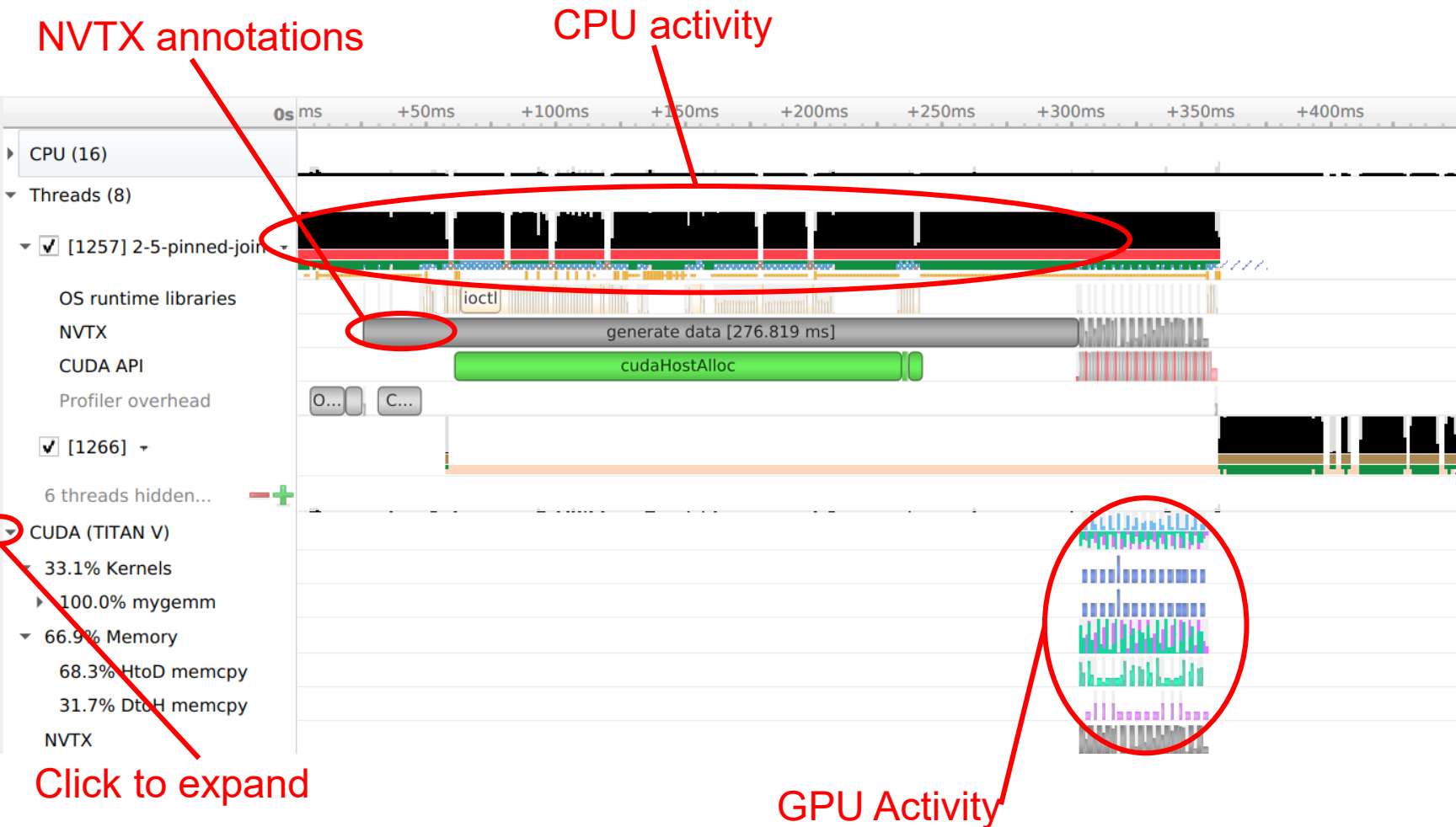
```
$ nsys profile. \ <nsight systems cli>  
-o output_file \ <record file>  
executable     <executable to run>
```

First Look

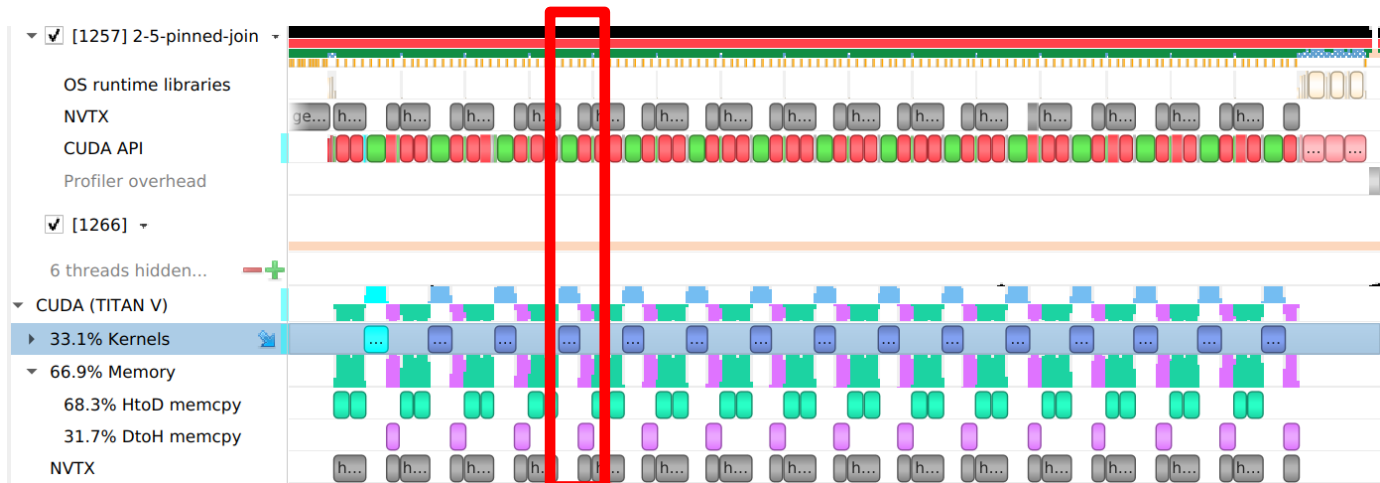
- File > Open > output_file.qdrep
- If multiple files are open, shown in the left pane
- Timeline of OS calls, CUDA calls, NVTX spans, and GPU activity



Timeline View



Click and
drag to
zoom in



Mouse over spans
for more info





GPU Teaching Kit



The GPU Teaching Kit is licensed by NVIDIA and the University of Illinois under the [Creative Commons Attribution-NonCommercial 4.0 International License](https://creativecommons.org/licenses/by-nc/4.0/).